

CinemaApp

CinemaApp is a full-stack cinema ticket booking system consisting of a Java Spring Boot backend and a React Native (Expo) mobile frontend. The application provides features such as searching for movies, choosing cinema halls, booking tickets, securing authentication, and managing user profiles.

Project Structure

The repository is organized into three main directories:

1. BackendCinema (Backend API)

A robust RESTful API built with Java 21 and Spring Boot 3.

- **Tech Stack:** Spring Boot (Web, Data JPA, Security, Validation), PostgreSQL, JWT for Auth, MapStruct for DTO mapping, and Lombok.
- **Database:** PostgreSQL configuration is provided via Docker Compose (`compose.yaml`).
- **Core Entities:** Movies, Cinemas, Halls, Seats, Screenings, Users, Ticket Types, Reservations, Tickets, and Promotions.
- **Key Features:**
 - JWT token-based authentication.
 - Endpoints mapped for managing movies, cinemas, bookings, and user profiles.
 - Docker support via `spring-boot-docker-compose`.

2. Frontend/CinemaApp (Mobile App)

A cross-platform mobile application developed with React Native and Expo, utilizing the **MVVM (Model-View-ViewModel)** architectural pattern to cleanly separate UI from business logic.

- **Tech Stack:** Expo (~54.0), React (19.1), React Native, React Navigation (Bottom Tabs, Native Stack), React Native Paper (UI Components), and TanStack React Query.
- **MVVM Architecture Details:**
 - **Model:** Handles the remote data layer, representing API DTOs (Data Transfer Objects) and network fetch requests (e.g., Axios/Fetch calls interacting with `/movies`, `/bookings`).
 - **ViewModel:** Managed via custom hooks and **TanStack React Query**. It acts as the bridge connecting the Model to the View. It maintains loading, error, and caching states, and transforms backend data into formats readily consumed by the View, keeping complex data logic out of UI components.
 - **View:** The React Native UI components. The views stay purely declarative and stateless regarding operations—they simply observe and react to the state provided by the ViewModel (hooks) and dispatch user interactions (like button clicks) back to it.
- **Features:** Consumes the REST API to provide user authentication, movie browsing, dynamic seat selection for screenings, and reservation management.
- **Project Structure:** Follows Expo Router file-based routing with app directories containing screens and configurations. API integration details and expected endpoints are documented in `Frontend/backend_structure.txt`.

Getting Started

Running the Backend

1. **Prerequisites:** Java 21, Maven, Docker (for the database).
2. Navigate to `BackendCinema`.
3. Start the database using Docker:

```
docker-compose up -d
```

4. Run the Spring Boot application:

```
./mvnw spring-boot:run
```

Running the Frontend

1. **Prerequisites:** Node.js, npm or yarn.
2. Navigate to `Frontend/CinemaApp`.
3. Install dependencies:

```
npm install
```

4. Start the Expo development server:

```
npx expo start
```

API Overview

The API is protected using JSON Web Tokens (JWT). Roles include standard Users and possibly Admin configurations. The endpoints are strictly typed with comprehensive Request and Response DTOs, including:

- `/auth/register` & `/auth/login` for JWT authentication

- /movies, /cinemas, /promotions for catalog browsing
- /movies/{movieId}/screenings and /screenings/{id}/seats for screening info
- /bookings for handling ticket reservations and payments
- /users/me for user profile and ticket history management
- uploads/{filename} for serving uploaded files